অধ্যায় ৩০

JavaScript Professional Mastery, Full-Stack Capstone এবং Career Roadmap

শেখার লক্ষ্য

এই অধ্যায় শেষে পাঠক (১) end-to-end product requirement থেকে implementation plan তৈরি করতে, (২) JavaScript frontend/backend/data/security একসঙ্গে ভাবতে, (৩) tested booking domain লিখতে, (৪) API contract ও architectural trade-off বুঝতে, (৫) deployment-readiness checklist বানাতে এবং (৬) নিজের portfolio ও interview roadmap সাজাতে পারবেন।

৩০.১ JavaScript Mastery মানে কী?

শুধু syntax মনে রাখা mastery নয়। Mastery হলো: **correctness, maintainability, security, observability, performance, accessibility, product thinking**—এগুলো মিলিয়ে বাস্তব সমস্যা সমাধান করা। একজন professional developer প্রশ্ন করেন: *ব্যবহারকারী কে? ব্যর্থ হলে কী হবে? data কে বদলাতে পারবে? duplicate request এলে কী হবে? পরে কীভাবে debug করব?*

স্তর	প্রধান দক্ষতা	প্রমাণ
Foundation	Variables, functions, objects, async	ছোট utility ও unit tests
Application	DOM, API, validation, state	ব্যবহারযোগ্য UI এবং error states
Backend	HTTP, auth, database	documented API ও integration tests
Engineering	Types, tests, security, CI	reproducible build ও code review
Architecture	scale, observability, failure modes	ADR, diagram, incident plan
Professional	collaboration, communication	portfolio, README, decisions

৩০.২ Capstone: TravelMaster Pro

ধরা যাক একটি travel booking service-এ ব্যবহারকারী package দেখবে, account তৈরি করবে, login করবে, booking দেবে, নিজের booking দেখবে এবং cancel করবে। সিস্টেমকে ভুল input, unauthorized request, duplicate submission এবং seat oversell থেকে রক্ষা করতে হবে। এই অধ্যায়ের runnable reference হলো **TravelMaster Pro**, `source-code/chapter-30/` -এ।

Functional requirements

1. Public catalog-এ package name, price (BDT) এবং available seats দেখাতে হবে।

2. Registration-এ নাম, email ও password validate করতে হবে।
3. Login-এর পরে short-lived bearer session token পাওয়া যাবে।
4. Booking-এর দাম **server-side package price** থেকে হিসাব হবে; client-sent price বিশ্বাস করা যাবে না।
5. Idempotency key দিয়ে একই booking request পুনরায় এলে দ্বিতীয়বার seat কমবে না।
6. নিজের booking দেখা ও cancel করা যাবে; অন্যের booking access করা যাবে না।
7. Cancel করলে একবারই seat ফেরত যাবে।
8. Tests-এ success, failure ও edge cases থাকবে।

Non-functional requirements

- Sensitive error-এ password বা internal stack trace ফেরত নয়।
- JSON body size সীমিত; invalid JSON-এ 400।
- Token মেয়াদ শেষ হলে 401।
- HTTP responses-এ consistent `{data}` অথবা `{error:{code,message}}` contract।
- Code-এ domain ও transport layer আলাদা।
- README-তে setup, limitations ও production gaps স্পষ্ট।

৩০.৩ Architecture — বাস্তব আর educational scope আলাদা রাখুন

```
Browser / Mobile Client
        |
        | HTTPS + JSON + bearer token
        v
   HTTP Router / Controller
        |
        v
    Domain Service -------> Audit events
        |
        +----> Users / Sessions
        +----> Package Inventory
        +----> Bookings / Idempotency

Educational reference: সব state একটি process-এর Map-এ
Production target: SQL database + transactional inventory + durable sessions
```

এই diagram-এর *production target* বাস্তবে implement করা হয়নি। Project-এ Node.js built-in `http` দিয়ে transport layer এবং `TravelMaster` class দিয়ে domain layer আলাদা করা হয়েছে। Frontend UI-ও এই runnable reference-এর অংশ নয়; নিচের section-এ frontend integration-এর design দেওয়া আছে।

৩০.৪ Project structure

```
source-code/chapter-30/
├── package.json
├── README.md
├── demo.mjs
├── src/
│   ├── domain.mjs
│   └── server.mjs
└── test/
    └── capstone.test.mjs
```

Node.js 20+ দিয়ে চালান:

TERMINAL

```
cd source-code/chapter-30
npm test
npm run check
npm run demo
npm start
```

Dependencies install করতে হবে না। `npm start`-এর default address `http://127.0.0.1:3000`। API example:

TERMINAL

```
curl http://127.0.0.1:3000/api/packages
curl -X POST http://127.0.0.1:3000/api/register \
  -H 'Content-Type: application/json' \
  -d '{"name":"Demo User","email":"demo@example.com","password":"long-password-123"}'
```

Login response থেকে token নিয়ে protected route-এ `Authorization: Bearer TOKEN` পাঠাতে হবে। Booking request-এর body:

JSON

```
{"packageId":"BT-4D","travelers":2,"idempotencyKey":"unique-key-001"}
```

API contract

Method	Endpoint	Auth	Expected result
GET	`/health`	No	200 health status
GET	`/api/packages`	No	200 catalog
POST	`/api/register`	No	201 new user
POST	`/api/login`	No	200 bearer token
POST	`/api/logout`	Token	200 session revoked (demo implementation does not validate token first)
POST	`/api/bookings`	Yes	201 booking

Method	Endpoint	Auth	Expected result
GET	`/api/bookings`	Yes	200 user's bookings
PATCH	`/api/bookings/:id/cancel`	Yes	200 cancellation

৩০.৫ Core JavaScript decisions

Validate at trust boundaries

`typeof travelers === 'number'` যথেষ্ট নয়: `NaN`, `Infinity`, fraction, negative value—সবই সমস্যা। `Number.isInteger(travelers) && travelers >= 1 && travelers <= 10` প্রয়োজন। Email regex এখানে basic format check, বাস্তব email ownership verification নয়।

Password handling

Plaintext password store করা যাবে না। Project-এ Node-এর `scryptSync` ও per-user random salt ব্যবহার হয়েছে। Production-এ calibrated Argon2id/scrypt parameters, credential-stuffing defense, login rate limiting এবং password-reset process প্রয়োজন। Authentication মানে কে login করেছে; authorization মানে সে **কোন data** access করতে পারবে।

Idempotency

একজন ব্যবহারকারী Submit button দুবার চাপলে দুইটি booking হওয়া উচিত নয়। Project-এ `(userId, idempotencyKey)` lookup করে একই payload-এ একই booking ফেরত দেওয়া হয়; ভিন্ন payload-এ 409। বাস্তব multi-instance server-এ এই guarantee দিতে database unique constraint এবং atomic transaction দরকার।

```javascript
const first = service.createBooking(userId, {
  packageId: 'BT-4D', travelers: 2, idempotencyKey: 'booking-001'
});
const retry = service.createBooking(userId, {
  packageId: 'BT-4D', travelers: 2, idempotencyKey: 'booking-001'
});
console.log(first.id === retry.id); // true
```

Inventory consistency

`if (seats >= requested) seats -= requested` একটি process-এ synchronous path-এ কাজ করে, কিন্তু একাধিক server/database transaction-এর ক্ষেত্রে race condition তৈরি করতে পারে। Production SQL উদাহরণ:

```sql
BEGIN;
UPDATE packages
SET seats = seats - $1
WHERE id = $2 AND seats >= $1
RETURNING id, price_bdt;
-- zero rows => sold out, ROLLBACK
INSERT INTO bookings(user_id, package_id, travelers, total_bdt)
VALUES ($3, $2, $1, $4);
COMMIT;
```

বাস্তবে booking insert, idempotency uniqueness, payment state এবং rollback—সব একই transactional design-এ নিশ্চিত করতে হবে। `$1`-style parameters SQL injection রোধের একটি গুরুত্বপূর্ণ অংশ।

৩০.৬ Frontend integration plan

একটি responsive UI-তে Home/Catalog, Package Details, Login/Register, Booking Checkout, My Bookings এবং Error/Empty States থাকবে। `fetch`-এর success path-এর পাশাপাশি loading, network failure, 401, 409 sold-out, duplicate submit, accessibility এবং keyboard navigation ভাবতে হবে।

```javascript
async function loadPackages() {
  const response = await fetch('/api/packages');
  if (!response.ok) throw new Error('Packages unavailable');
  const { data } = await response.json();
  return data;
}
```

Important: Exampleটি conceptual frontend integration; capstone archive-এ browser UI তৈরি বা test করা হয়নি। Token browser storage-এ রাখার সিদ্ধান্ত XSS threat model অনুযায়ী নিতে হবে। HttpOnly cookie নিলে CSRF protection-ও ডিজাইন করতে হবে।

৩০.৭ Testing pyramid এবং quality gates

- **Unit:** validation, price calculation, idempotency, seat restoration।
- **Integration:** HTTP register → login → booking → history।
- **Security-focused:** unauthenticated request 401, অন্যের booking 404, malformed JSON 400।
- **E2E (future):** real browser, mobile viewport, screen-reader behavior, production-like DB।

CI gate হিসেবে `npm run check` এবং `npm test` চালানো যায়। Tests pass করলেও full security audit বা production correctness প্রমাণ হয় না।

৩০.৮ Production architecture upgrade plan

1. **PostgreSQL:** users, packages, bookings, idempotency_requests, sessions, audit_events schema।
2. **Migrations:** version-controlled migrations; constraints, foreign keys, unique keys।

3. **Atomic transactions:** seat reservation ও booking creation একই transaction।
4. **Payments:** gateway webhook verification, replay protection, reconciliation, refunds।
5. **Session security:** durable session store, token rotation/revocation, HTTPS, rate limits।
6. **Observability:** request ID, structured logs, latency/error metrics, alerting।
7. **Delivery:** Docker, CI, staged rollout, backups, recovery drill, rollback।
8. **Privacy:** data minimization, retention policy, access control, secret management।

Never expose demo server directly to the public internet as a production travel-booking service.

৩০.৯ Architecture Decision Record (ADR)

ADR-001: In-memory domain for educational reference. Context: পাঠককে dependency ছাড়াই code run করাতে হবে। Decision: Node built-ins, small modular service, in-memory repositories। Benefit: সহজ test ও setup। Cost: restart-এ data হারায়, horizontal scaling নেই, persistent transaction নেই। Next decision: PostgreSQL + transactional repository। ADR লিখলে future developer বুঝবেন কেন একটি সিদ্ধান্ত নেওয়া হয়েছিল এবং কখন বদলাতে হবে।

৩০.১০ Final assessment rubric (100 marks)

ক্ষেত্র	Marks	মূল্যায়ন
Requirements ও user flows	10	edge cases স্পষ্ট
JavaScript design	15	modularity, validation
API & authentication	15	correct status, ownership
Data consistency	15	idempotency, inventory
Tests & debugging	15	automated coverage
Security & privacy	10	threats, mitigations
Deployment & monitoring plan	10	readiness checklist
Documentation & communication	10	README, ADR, demo

Score interpretation: 85–100 strong capstone; 70–84 good foundation; 50–69 targeted improvement; below 50 revise requirements and tests first। এটি learning rubric, official certification নয়।

৩০.১১ Professional roadmap — 12 weeks

Weeks	Focus	Deliverable
1–2	Fundamentals revision, async, Git	tested utility library
3–4	DOM, accessibility, API calls	responsive package catalog

Weeks	Focus	Deliverable
5–6	Node.js, auth, REST	booking API
7–8	SQL, transactions, Prisma/ORM	persistent booking flow
9–10	tests, TypeScript, security	CI-tested refactor
11	Docker, deploy, monitoring	staging demo
12	portfolio, README, interviews	case study + walkthrough

Portfolio-তে শুধু screenshot নয়: **problem, architecture, trade-offs, tests, limitations, what I would improve** লিখুন। Interview-এ event loop, closures, async errors, authorization, SQL transaction, caching, idempotency এবং system design নিয়ে যুক্তি ব্যাখ্যা করার অভ্যাস করুন।

৩০.১২ দশটি অনুশীলন ও সমাধান নির্দেশনা

1. **Email normalization:** `TEST@EXAMPLE.COM` দিয়ে duplicate account কেন reject হয়? **সমাধান:** `trim().toLowerCase()`-এর পরে `usersByEmail` lookup।
2. **Price tampering:** Client `{price:1}` পাঠালে total কত? **সমাধান:** catalog-এর `price * travelers`; client price উপেক্ষিত।
3. **Retry:** একই idempotency key ও payload দুইবার পাঠান। **সমাধান:** একই booking ID, একবার seat decrement।
4. **Key collision:** একই key-তে travelers বদলান। **সমাধান:** 409 `KEY_CONFLICT`।
5. **Unauthorized access:** token ছাড়া `/api/bookings`। **সমাধান:** 401 `UNAUTHORIZED`।
6. **Ownership:** user A-এর booking user B cancel করতে পারে? **সমাধান:** না, 404।
7. **Cancellation:** একই booking দুবার cancel করলে seats কতবার বাড়বে? **সমাধান:** একবার।
8. **Expiry:** clock এক ঘণ্টা এগিয়ে দিলে token valid? **সমাধান:** `expiresAt <= now` হলে 401।
9. **Database design:** booking-এ foreign key কী হবে? **সমাধান:** `user_id → users.id`, `package_id → packages.id`; constraints ও indexes।
10. **Operational readiness:** deployment-এর আগে পাঁচটি কাজ লিখুন। **সমাধান:** persistent DB, transactions, auth hardening, rate limits, backups/monitoring।

৩০.১৩ দশটি MCQ — উত্তর ও ব্যাখ্যা

1. **Server-side price কেন?** A. UI দ্রুত B. tampering প্রতিরোধ C. CSS D. caching। **উত্তর B:** client input বিশ্বাসযোগ্য নয়।
2. **Idempotency key-এর উদ্দেশ্য?** A. SEO B. duplicate side effect রোধ C. image resize D. login। **উত্তর B।**
3. **Authentication বনাম authorization?** A. একই B. identity বনাম permission C. CSS বনাম HTML D. network বনাম disk। **উত্তর B।**
4. **Atomic booking-এর জন্য কী দরকার?** A. `setTimeout` B. DB transaction C. localStorage D. alert। **উত্তর B।**

5. **401 কখন?** A. login প্রয়োজন B. package নেই C. sold out D. server success। **উত্তর A।**
6. **409 কোন ক্ষেত্রে যুক্তিযুক্ত?** A. duplicate/conflicting request B. healthy server C. logout success D. list packages। **উত্তর A।**
7. **Unit test-এর কাজ?** A. পুরো cloud deploy B. ছোট behavior isolate করা C. DNS বদলানো D. design logo। **উত্তর B।**
8. **ADR কী document করে?** A. animation B. architecture decision/trade-off C. only CSS D. invoice। **উত্তর B।**
9. **In-memory data-এর সীমাবদ্ধতা?** A. restart-এ state হারায় B. automatically replicated C. ACID guarantee D. infinite scale। **উত্তর A।**
10. **Portfolio case study-তে কী জরুরি?** A. শুধু screenshot B. problem, decisions, evidence, limitations C. শুধু color D. শুধু title। **উত্তর B।**

৩০.১৪ Final project extension challenges

Beginner: catalog filtering ও pagination। **Intermediate:** PostgreSQL persistence, Prisma migration, secure auth session। **Advanced:** payment webhook, reservation expiry, queue-based notifications, concurrency tests, observability। **Expert:** multi-region trade-offs, disaster recovery, load test, incident simulation। প্রতিটি feature-এ requirement → threat model → test → implementation → review → documentation flow অনুসরণ করুন।

৩০.১৫ বই শেষ, শেখা নয়

এই ৩০ অধ্যায়ের উদ্দেশ্য হলো আপনাকে independent engineering decisions নিতে সাহায্য করা। প্রতিটি নতুন technology শেখার আগে নিজেকে জিজ্ঞেস করুন: *এটি কোন সমস্যার সমাধান? এর ব্যর্থতার ধরন কী? আমি কীভাবে test করব?* এই mindset-ই long-term JavaScript mastery-এর ভিত্তি।